Literature Review
P2P File Synchronization System

Action Learning Project

Backend (Go, C++)

Arihant Jain
Su Huizhe

Frontend and CI/CD (Java, GitHub Actions)

Nizar Soufiya
Shengkang Duan

May 29, 2026

Contents

1 Problem Context

1.1 Scenario and Pain Points

To motivate the problem, we begin with a concrete collaborative scenario commonly observed in university projects.

University students frequently work in temporary groups and need to exchange files efficiently without relying on complex infrastructure, cloud account management, or external storage devices. During the course of a project, team members collaboratively produce and modify heterogeneous files, including source code, documents, images, datasets, and presentation materials. Since collaborators often use different operating systems and development environments, cross-platform compatibility further increases the complexity of file sharing and coordination.

In practice, temporary student teams usually lack the time, resources, and technical expertise required to maintain a dedicated file management or synchronization system. As a result, files are often exchanged through ad hoc communication channels such as group chats or email attachments. This approach introduces several issues, including duplicated files, inconsistent versions, lack of synchronization, and absence of proper change tracking.

In addition, ownership and persistence of project data become problematic once the collaboration ends. Some members may wish to preserve the project files for future reference, while others may prefer to discard them. However, because the files are typically distributed informally and managed inconsistently, long-term access often depends on a single individual maintaining the archive. This creates risks of data loss, incomplete project history, and fragmented ownership.

Therefore, the students require a lightweight and decentralized solution that enables them to:

- create a shared repository without complicated server deployment or administrative setup;
- support synchronization of heterogeneous file types across multiple platforms;
- track modifications and maintain version history;
- provide shared ownership of project data among collaborators; and
- avoid dependence on a single participant for long-term storage, archival, or deletion of the project files.

1.2 Problem Definition

Based on the scenario described above, small-scale collaborative groups require a file-sharing mechanism that is lightweight, decentralized, and easy to use. In such environments, participants mainly care about whether they can quickly form a shared workspace, exchange project files reliably, preserve previous versions when changes are made, and continue working with the shared folder as if it were a normal local directory.

From this objective, the problem can be defined as the design of a peer-to-peer directory-level synchronization system with lightweight version control. More specifically, the system must solve the following technical problems:

- **Local peer discovery and dynamic membership.** The system must discover available peers on the same local network without relying on a central server. It should also support dynamic participation, where peers may join, leave, or temporarily disconnect during collaboration.

- **Directory-level change detection and propagation.** The system must monitor a shared local directory and detect file creation, modification, and deletion events. These changes should be propagated to other online peers without requiring users to manually upload, download, or explicitly send files.
- **Rapid synchronization among online peers.** When a change occurs, the system should synchronize the updated state to other available peers within a short delay, so that collaborators can observe a consistent project workspace during active collaboration.
- **Support for heterogeneous file types and large files.** The system must handle different categories of project files, including text files, source code, documents, images, and binary files. For large files, the system should support efficient transfer mechanisms rather than assuming that every file can be sent as a single small message.
- **Simple versioning and rollback.** Before a file is overwritten by a synchronized update, the system should preserve an older copy or historical state. This provides a basic recovery mechanism against accidental overwrites, incorrect updates, or unintended file changes.
- **Transparent integration with the local file system.** The shared workspace should behave like a normal directory on the user's computer. Users should be able to create, edit, move, and delete files using their usual tools, while synchronization and version management are handled by the system in the background.

Therefore, the central technical problem is to provide a serverless, peer-to-peer, directory-level synchronization mechanism that supports dynamic local-network collaboration, heterogeneous file transfer, rapid update propagation, and basic version recovery while preserving the simplicity of a normal local folder.

2 Overview of the Existing Work

TODO (team): summarize existing work and systems. this is included

3 Comparison of Approaches

We compare collaborative file synchronization tools by setup effort, server requirements, and capability to synchronize heterogeneous files across local meshes [1–6].

3.1 Evaluation of Existing Paradigms

1. **Cloud Directory Services** (e.g., Google Drive [1], Dropbox [2], Nextcloud):
   These services rely on centralized cloud infrastructure. While they offer simple setup and ease of use, they suffer from high transfer latency, storage volume quotas, and strict dependency on external internet connectivity. Centralized hosting also raises user data privacy concerns and imposes access permissions that create collaboration bottlenecks for temporary teams.
2. **Version Control Systems** (e.g., Git [3, 4], SVN):
   VCS tools track repository history using directed acyclic graphs (DAGs). Although they provide powerful conflict resolution, change logs, and branch-level safety, they are designed for text-based source code. They do not handle large binary files well and require users to

manually run staging, committing, and remote push/pull cycles. This workflow introduces steep learning curves for non-technical team members.

3. **P2P File Synchronization** (e.g., Syncthing [5], Resilio Sync [6]):
   Decentralized sync programs mirror directories directly between devices without central servers, avoiding cloud storage quotas and allowing high-speed local transfers. However, they lack integrated version rollback histories, making accidental file overwrites difficult to undo. Setup also requires manual node sharing and folder approvals, which adds configuration friction for short-term projects.

3.2 Key Product Differences: Syncthing vs. Resilio Sync

We analyze the differences between the leading peer-to-peer synchronization tools to clarify our design choices:

- **Architecture and Openness:** Syncthing is open-source and uses the public Block Exchange Protocol (BEP). It enforces a strict security model where devices must exchange long cryptographic IDs and mutually approve folder invitations. Resilio Sync is proprietary, based on the BitTorrent protocol, and supports selective sync, but has commercial license limits and proprietary lock-in.
- **Lightweight Collaboration Gap:** Both tools act as generic directory mirrors rather than project-specific workspaces. They lack automated, non-destructive version histories that team members can navigate easily. If a peer saves a broken draft, that file immediately overwrites the state of all other online peers, requiring complex administrative configuration to recover.

3.3 Proposed P2P Synchronization System

To address these limitations, we propose a serverless local directory synchronization system. It combines the direct transfer speed of P2P mirroring with the version safety of a lightweight revision tool. The proposed system features:

- **Zero Configuration (planned):** Peer discovery relies on Multicast DNS (mDNS) [7] and DNS-SD [8] broadcasts on the local subnet, forming a sync mesh without manual IP or node ID exchange.
- **Local Socket Transfer (planned):** TCP connections [9] handle data replication directly between peers over the local network, avoiding external server roundtrips.
- **Autonomous Coordination (planned):** Every node acts as an independent repository coordinator, allowing concurrent edits that propagate without administrative approval bottlenecks.
- **Durability and Recovery (planned):** We combine a Last-Write-Wins (LWW) conflict policy with automated local backups before any overwrite. This avoids the implementation complexity of vector clocks or CRDTs for small-scale student teams (3–5 peers).

3.4 Core Project Hypotheses

We formulate the following testable scientific hypotheses to guide the evaluation of our proposed system:

- **Primary Hypothesis (H_1):** A localized, serverless synchronization architecture using mDNS/DNS-SD discovery and direct TCP socket replication can achieve sub-second average synchronization latency on standard local area networks (WLAN) without reliance on external cloud infrastructure.
- **Secondary Hypothesis (H_2):** Utilizing a single logical timestamp (Lamport clock) counter per file, paired with a deterministic Last-Write-Wins (LWW) resolution rule and local file backups, provides sufficient consistency and rollback recovery for small-scale collaborative teams (3-5 peers) while avoiding the state and computational overhead of vector clocks or CRDTs.

3.5 Structured Comparison Table

Table 1 evaluates the feature tradeoffs of the three paradigms.

Table 1: Comparison of Collaborative File Sharing and Synchronization Approaches

Feature / Dimension	**Cloud Directory Services**	**Version Control Systems**	**P2P File Sync (Proposed)**
Primary Topology	Centralized Client-Server	Distributed (DAG-based)	Peer-to-Peer (Mesh)
Network Discovery	Centralized cloud portal	Remote repository server	Local mDNS / LAN broadcast
Setup Complexity	Account setup	Repository creation / auth	Low (auto-discovery planned)
Target Use Case	Generic office documents	Source code tracking	Live directory mirroring
Conflict Resolution	Duplicate file generation	Manual merges / diff resolution	LWW (planned)
Version Control	Periodic file revisions	Explicit commits and branches	Local backups (planned)
File Watcher	Periodic polling	Manual staging	Kernel events (planned)
Bandwidth Efficiency	Server upload/download	Incremental compression diffs	Direct sockets / planned delta sync
Internet Dependency	High (cloud servers required)	High (for pull/push)	None (offline-capable)
Data Privacy	Low (third-party storage)	Medium (third-party servers)	High (local-only storage)
Access Permissions	Centralized ACL approvals	Branch rules and PR reviews	Peer-to-peer access

4 Identified Gap

4.1 Limitations of Centralized Solutions

TODO (team): add limits of centralized solutions.

4.2 Limitations of Existing Peer-to-Peer Solutions

TODO (team): add limits of existing P2P solutions.

4.3 The Gap

TODO (team): describe the gap.

5 Positioning Your Project

5.1 Key Differentiators

TODO (team): add key differentiators.

5.2 Design Philosophy

TODO (team): add design philosophy.

6 Research to Application Mapping

To translate academic theories into our proposed architecture, we map eight core research insights to planned components in our Go/C++ design:

1. **Zero-Configuration Peer Discovery on LAN**
    - **Research Insight:** Cheshire and Krochmal [7,8] establish that Multicast DNS (mDNS) and DNS-Based Service Discovery (DNS-SD) enable serverless local service registration and resolution using standard IP multicast queries.
    - **Application in Project:** We plan a Go-based peer discovery component that broadcasts service records under the `_p2psync._tcp` domain and dynamically browses the local subnet to resolve peer addresses, removing the need for manual configuration.
2. **Causal Ordering of Concurrent Updates**
    - **Research Insight:** Lamport [10] demonstrates that logical clocks and causal relation tracking can order events in a distributed system without relying on synchronized physical clocks.
    - **Application in Project:** We plan to attach per-file logical timestamps (Lamport clocks) to track change sequences and detect concurrent modifications, avoiding the state and coordination overhead of vector clocks for the initial release.
3. **Consistency Strategies under Network Partitions**
    - **Research Insight:** Gilbert and Lynch's proof of the CAP Theorem [11] establishes that distributed systems must trade consistency for availability during network partitions.

- **Application in Project:** We choose availability by allowing peers to make local modifications offline. When connection is restored, a resync process runs to resolve conflicts and converge on a common state.

4. **Conflict Handling and Durability**

   - **Research Insight:** Replicated storage systems like Bayou [12] demonstrate that resolving conflicts requires deterministic policies paired with non-destructive version backups to ensure data durability.
   - **Application in Project:** We plan to use a Last-Write-Wins (LWW) policy based on logical timestamps. Before overwriting a local file, we plan to store a backup copy to protect against data loss.

5. **Conflict-Free Convergence Evolution**

   - **Research Insight:** Shapiro et al. [13] prove that Conflict-Free Replicated Data Types (CRDTs) allow concurrent updates to merge automatically and converge mathematically.
   - **Application in Project:** We plan to use LWW for the initial release to reduce complexity, but construct our SQLite metadata schema and Go coordination logic to support state-based CRDTs in future versions.

6. **Bandwidth-Efficient Synchronization**

   - **Research Insight:** Tridgell and Mackerras [14] show that rolling checksums allow remote directories to synchronize by transferring only modified blocks instead of entire files.
   - **Application in Project:** We plan to start with whole-file transfers for simplicity. In a later optimization phase, we will add block-level diffing to minimize local WLAN bandwidth use.

7. **Kernel-Level File Watching**

   - **Research Insight:** Operating system event APIs like Linux's inotify [15] and macOS's FSEvents [16] provide kernel-level event streams for file additions, modifications, and deletions without the overhead of periodic polling.
   - **Application in Project:** We plan a C++ file watcher daemon that uses native OS APIs (inotify and FSEvents) to detect directory events with minimal CPU overhead, notifying the Go coordination engine via IPC.

8. **Data Integrity and Reliable Transport**

   - **Research Insight:** Cryptographic hashing standard SHA-256 [17] provides unique content identifiers that verify data integrity, while the Transmission Control Protocol (TCP) [9] guarantees reliable, ordered packet delivery.
   - **Application in Project:** We plan to let the C++ daemon hash local files for change verification, and let the Go network coordinator transfer files over TCP sockets to ensure error-free replication.

7 Key References

The following six publications serve as the primary theoretical and technical foundation for our proposed system's architecture and design choices.

1. **RFC 6762 & RFC 6763 (Stuart Cheshire and Marc Krochmal, 2013)** [7, 8]
   These specifications define Multicast DNS (mDNS) and DNS-Based Service Discovery (DNS-SD), which form the basis for zero-configuration networking. They demonstrate how local devices can advertise and resolve services without a centralized DNS server. We use these protocols to plan our Go-based peer discovery component, allowing team members on the same local network to connect automatically and sync directories without manual setup.

2. **Leslie Lamport (1978) - "Time, Clocks, and the Ordering of Events in a Distributed System"** [10]
   This paper introduces logical clocks to track event ordering and causality in distributed systems where physical clocks cannot be synchronized. It provides the foundation for our synchronization logic. We plan to attach a single logical timestamp counter to each file metadata entry. This enables our Go coordinator to detect concurrent modifications and resolve update sequencing chronologically without the complexity of full vector clocks.

3. **Seth Gilbert and Nancy Lynch (2002) - "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services"** [11]
   This work provides the formal proof of the CAP Theorem, showing that a distributed system cannot guarantee consistency, availability, and partition tolerance simultaneously. This theorem guides our architectural choice to prioritize availability. We allow peers to edit files locally while offline and reconcile changes eventually upon reconnection, which fits the dynamic connection cycles of student teams.

4. **Marc Shapiro et al. (2011) - "A Comprehensive Study of Convergent and Commutative Replicated Data Types"** [13]
   This research report analyzes CRDTs, demonstrating how data structures can achieve eventual consistency mathematically without consensus protocols. While we adopt a simpler Last-Write-Wins policy for our initial release to reduce complexity, this study serves as our design roadmap, ensuring our SQLite schema and Go metadata models can support state-based CRDTs in future iterations.

5. **Andrew Tridgell and Paul Mackerras (1996) - "The Rsync Algorithm"** [14]
   This paper presents a rolling-checksum algorithm that synchronizes directories by transferring only modified file blocks. It forms the basis of our planned bandwidth optimization. In our design, we begin with whole-file transfer pipelines, and plan to integrate this block-level delta sync in a later phase to optimize local WLAN usage for large binary assets.

6. **Karin Petersen et al. (1995) - "Managing Update Conflicts in Bayou, a Weakly Connected Replicated Storage System"** [12]
   This paper explores conflict management and reconciliation in weakly connected replicated storage systems. It introduces conflict detection and resolution policies, informing our decision to combine an automated Last-Write-Wins (LWW) resolution rule with local backup preservation. This ensures predictable reconciliation while keeping older file versions accessible to prevent accidental data loss.

8 Conclusion

8.1 Literature Review Synthesis

Our analysis of distributed systems literature highlights that building a serverless, local directory sync tool requires combining data-integrity algorithms with network and clock synchronization standards. mDNS and DNS-SD [7, 8] remove the need for central registry servers, while logical clocks [10] establish event ordering in decentralized networks. Because partitions are expected on local subnets, the CAP Theorem [11] suggests prioritizing availability and eventual convergence. Deterministic resolution policies combined with version backups [12] provide a safe conflict-handling baseline, with CRDTs [13] offering a path toward automatic conflict-free convergence.

8.2 Influence on Design

The literature shapes our proposed Go/C++ architecture:

- We plan to use mDNS/DNS-SD (via Go) to automate local network peer discovery, removing setup friction.
- We plan to use logical timestamps (Lamport clocks) and a Last-Write-Wins conflict resolution policy to manage concurrent edits predictably.
- We plan to favor local availability, ensuring users can work offline and sync differences upon reconnection.
- We plan a split where Go handles network coordination and C++ handles OS-level file watching (inotify [15], FSEvents [16]) and hashing (SHA-256 [17]) over reliable TCP sockets [9].

8.3 Limitations and Future Work

While our current design provides a synchronization baseline, we recognize limitations in bandwidth usage and conflict handling. Whole-file transfers are simple to implement but inefficient for large assets. Our next steps are:

- Implement the Go-to-C++ socket handover pipeline to connect the network and storage layers.
- Implement the rsync-style rolling checksum algorithm [14] in C++ to enable delta transfers.
- Introduce state-based CRDT schemas [13] in Go to support automatic, conflict-free metadata merging.

References

[1] Google, "Google drive help: Use drive for desktop," https://support.google.com/drive/, 2026, accessed: 2026-05-28.

[2] Dropbox, "Dropbox help center," https://help.dropbox.com/, 2026, accessed: 2026-05-28.

[3] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014.

[4] GitHub, "Github docs: About repositories," https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories, 2026, accessed: 2026-05-28.

[5] S. Community, "Syncthing documentation," https://docs.syncthing.net/, 2026, accessed: 2026-05-28.

[6] I. Resilio, "Resilio sync documentation," https://help.resilio.com/hc/en-us, 2026, accessed: 2026-05-28.

[7] S. Cheshire and M. Krochmal, "Multicast dns," RFC 6762, 2013, accessed: 2026-05-28. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6762

[8] ——, "Dns-based service discovery," RFC 6763, 2013, accessed: 2026-05-28. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6763

[9] W. M. Eddy, "Transmission control protocol (tcp)," RFC 9293, 2022, accessed: 2026-05-28. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9293

[10] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[11] E. A. Brewer, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.

[12] K. Petersen, M. Spreitzer, D. Terry, M. Theimer, and A. Demers, "Managing update conflicts in bayou, a weakly connected replicated storage system," in *Proceedings of the 15th ACM Symposium on Operating Systems Principles*, 1995, pp. 172–182.

[13] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "A comprehensive study of convergent and commutative replicated data types," *INRIA Research Report*, no. RR-7506, 2011.

[14] A. Tridgell and P. Mackerras, "The rsync algorithm," in *Proceedings of the USENIX Annual Technical Conference*, 1996.

[15] M. Kerrisk, "inotify(7) — linux manual page," https://man7.org/linux/man-pages/man7/inotify.7.html, 2024, accessed: 2026-05-28.

[16] Apple, "File system events programming guide," https://developer.apple.com/library/archive/documentation/Darwin/Conceptual/FSEvents_ProgGuide/Introduction/Introduction.html, 2007, accessed: 2026-05-28.

[17] N. I. of Standards and Technology, "Fips pub 180-4: Secure hash standard (shs)," 2015, accessed: 2026-05-28. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf